

Implémentation et explication d'un modèle de régression logistique en Python

Rapport de projet

Étudiant	OUEDRAOGO Wend'Yida Jalil Dylan
Enseignante	Raquel URENA
Formation	Licence 3 Mathématiques-Informatique
Année universitaire	2025-2026
Lien GitHub public	https://github.com/jaliloued-byte/projet-regression.git

Résumé

Ce rapport présente l'implémentation complète d'une régression logistique codée depuis zéro en Python. Le modèle est appliqué à un problème de classification binaire : prédire l'admission universitaire d'un étudiant à partir de variables simples. L'étude détaille la motivation du problème, les algorithmes existants, la méthodologie implémentée, les résultats obtenus et les limites du modèle.

Sommaire

1. Introduction
 2. Motivation du problème
 3. Algorithmes existants
 4. Méthodologie implémentée
 5. Résultats expérimentaux
 6. Challenges rencontrés
 7. Organisation du travail et GitHub
 8. Conclusion
- Bibliographie

1. Introduction

La classification binaire est une tâche fréquente en informatique décisionnelle et en apprentissage statistique. Elle consiste à classer chaque observation dans l'une des deux catégories possibles. Dans ce projet, le problème étudié est la prédiction de l'admission universitaire d'un étudiant à partir de quelques variables simples : sa note à un test, le taux de réussite de son lycée et sa participation ou non à des activités extra-scolaires.

L'objectif principal est de comprendre le fonctionnement de la régression logistique en l'implémentant manuellement. Ce modèle est bien adapté à ce type de problème, car il permet d'obtenir une probabilité, puis de prendre une décision à partir d'un seuil. Il permet aussi d'interpréter l'influence des différentes variables grâce aux coefficients obtenus. L'implémentation a été faite en Python, sans utiliser de bibliothèque spécialisée en apprentissage automatique.

Le rapport présente d'abord le contexte et l'intérêt du problème. Il aborde ensuite les principales méthodes utilisées pour la classification binaire, puis décrit la méthode choisie, les résultats obtenus, les difficultés rencontrées et l'organisation du travail dans le dépôt GitHub public.

2. Motivation du problème

La régression logistique permet de répondre à une question simple : quelle est la probabilité qu'un élément appartienne à la classe positive ? Contrairement à une régression linéaire classique, le résultat obtenu est toujours compris entre 0 et 1, grâce à la fonction sigmoïde. C'est pour cela que ce modèle est bien adapté aux problèmes où la réponse finale est binaire.

Dans ce projet, l'objectif est de prédire si un étudiant est admis ou non. Le problème a volontairement été simplifié pour se concentrer sur le fonctionnement du modèle. Les variables choisies sont faciles à comprendre : une meilleure note au test, un taux de réussite plus élevé au lycée ou encore la participation à une activité extra-scolaire peuvent augmenter les chances d'admission.

Le projet permet de mieux comprendre plusieurs notions importantes, comme les probabilités, la fonction de coût, la descente de gradient, l'évaluation du modèle et l'interprétation des coefficients. Ces notions sont essentielles pour bien comprendre les modèles de classification.

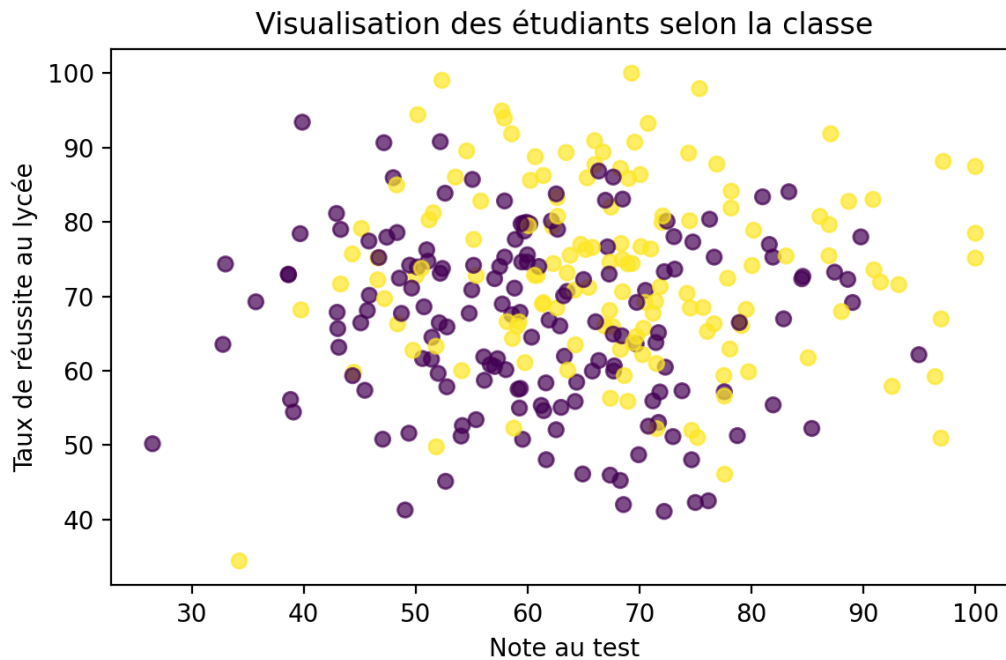


Figure 1 - Répartition visuelle des étudiants selon les deux variables numériques.

3. Algorithmes existants

Plusieurs méthodes peuvent être utilisées pour résoudre un problème de classification binaire. Certaines sont très simples et interprétables, tandis que d'autres sont plus complexes et potentiellement plus performantes.

- La régression logistique : modèle linéaire qui prédit une probabilité avec une fonction sigmoïde.
- Les arbres de décision : modèles interprétables reposant sur une succession de règles de décision.
- Les k plus proches voisins : méthode qui classe une observation selon les classes de ses voisins les plus proches.
- Les réseaux de neurones : modèles plus flexibles, mais plus difficiles à interpréter et plus lourds à entraîner.
- Les réseaux bayésiens : modèles probabilistes qui représentent les dépendances conditionnelles entre variables.

La régression logistique a été retenue car elle correspond directement au cahier des charges du sujet. Elle est suffisamment simple pour être codée depuis zéro, tout en étant assez riche pour illustrer l'entraînement par descente de gradient, l'analyse des performances et l'interprétation des coefficients.

4. Méthodologie implémentée

La méthodologie suivie reprend les étapes classiques d'un projet de classification : création du jeu de données, exploration, séparation entraînement-test, normalisation, entraînement du modèle, prédiction, évaluation et interprétation.

4.1 Jeu de données

Le jeu de données contient 300 observations artificielles. Chaque ligne représente un étudiant. Les variables explicatives sont 'note_test', 'taux_lycee' et 'extrasco'. La variable cible est 'admission', qui vaut 1 si l'étudiant est admis et 0 sinon.

4.2 Prétraitement

Les données sont séparées en deux ensembles : 80 % pour l'entraînement et 20 % pour le test. Les variables numériques sont ensuite normalisées avec la moyenne et l'écart-type calculés uniquement sur l'ensemble d'entraînement. Cette précaution évite une fuite d'information depuis l'ensemble de test.

4.3 Modèle mathématique

Le modèle calcule d'abord une combinaison linéaire des variables : $z = w^T x + b$. Cette valeur est ensuite transformée en probabilité par la fonction sigmoïde : $\sigma(z) = 1 / (1 + \exp(-z))$. La prédiction finale est obtenue en comparant cette probabilité à un seuil fixé à 0,5.

La fonction de coût utilisée est la log-loss. Elle pénalise fortement les prédictions très confiantes mais incorrectes. L'objectif de l'entraînement est de réduire cette fonction de coût.

4.4 Descente de gradient

Les paramètres du modèle sont les poids associés aux variables et le biais. Ils sont initialisés à zéro, puis mis à jour à chaque itération selon le gradient de la fonction de coût. Le taux d'apprentissage utilisé est 0,1 et le nombre d'itérations est fixé à 2000.

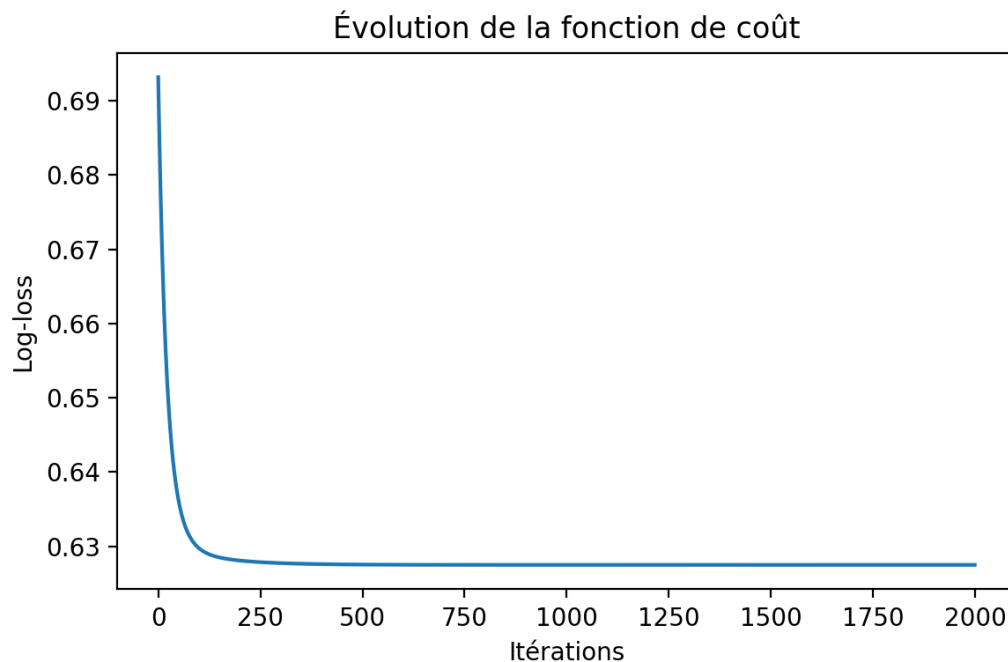


Figure 2 - Diminution de la log-loss pendant l'entraînement.

5. Résultats expérimentaux

Les résultats ont été calculés sur l'ensemble de test, qui contient 60 observations. Les métriques utilisées sont l'accuracy, la précision, le rappel, le F1-score et l'AUC. La matrice de confusion permet d'identifier plus précisément les erreurs du modèle.

Métrique	Valeur obtenue
Accuracy	0.7833
Précision	0.8333

Métrique	Valeur obtenue
Rappel	0.7576
F1-score	0.7937
AUC	0.8345
Temps d'entraînement	0.0895 seconde(s)
Log-loss finale	0.6275

La matrice de confusion obtenue est composée de 22 vrais négatifs, 5 faux positifs, 8 faux négatifs et 25 vrais positifs. Le modèle obtient une accuracy d'environ 78.33%, ce qui indique qu'il classe correctement une majorité des exemples de test.

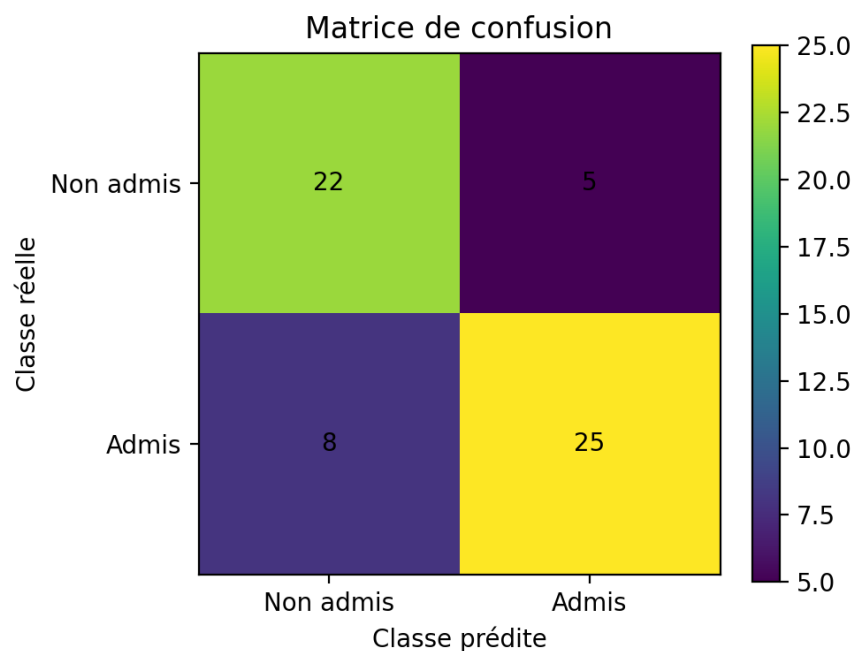


Figure 3 - Matrice de confusion obtenue sur l'ensemble de test.

La précision vaut 83.33%. Cela signifie que, parmi les étudiants prédits comme admis, une grande proportion est effectivement admise. Le rappel vaut 75.76%, ce qui mesure la capacité du modèle à retrouver les étudiants réellement admis. Le F1-score combine ces deux informations.

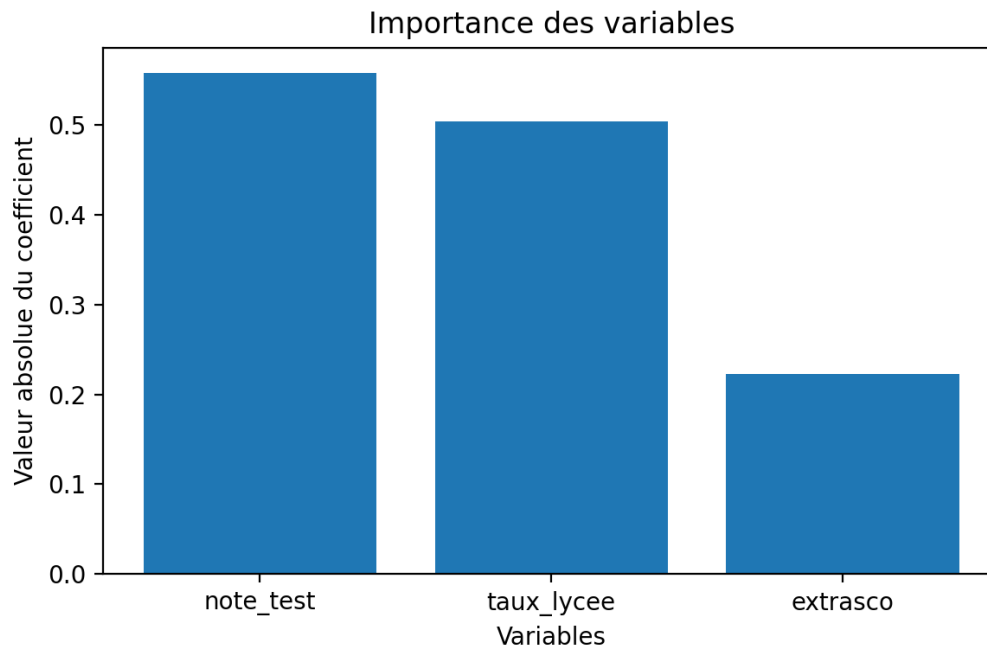


Figure 4 - Importance absolue des coefficients du modèle.

L'analyse des coefficients montre que les variables numériques normalisées ont l'influence la plus forte dans cette expérience. Les coefficients positifs indiquent que l'augmentation de ces variables augmente la probabilité d'admission. La variable 'extrasco' a également un effet positif, mais plus faible dans les résultats obtenus.

6. Challenges rencontrés

Le premier challenge a été de respecter la contrainte principale du projet : ne pas utiliser de bibliothèque de machine learning. Cela impose de coder soi-même la séparation des données, la normalisation, la fonction sigmoïde, la log-loss, la descente de gradient et les métriques d'évaluation.

Le deuxième challenge a été d'éviter les instabilités numériques. Par exemple, les valeurs extrêmes de la fonction exponentielle peuvent provoquer des dépassements. Pour limiter ce problème, l'entrée de la sigmoïde est bornée. De même, les probabilités utilisées dans la log-loss sont limitées entre epsilon et $1 - \text{epsilon}$ afin d'éviter le calcul de $\log(0)$.

Le troisième challenge concerne l'interprétation des résultats. Comme les données sont artificielles, les conclusions doivent rester prudentes. Le modèle sert principalement à démontrer le fonctionnement de la régression logistique et non à prédire un vrai processus universitaire.

7. Organisation du travail et GitHub

Le projet est organisé autour d'un notebook Jupyter principal. Celui-ci contient les explications, les fonctions codées, les résultats et les graphiques.

Le dépôt GitHub public contient l'ensemble de ces fichiers.

Lien GitHub public : <https://github.com/jaliloued-byte/projet-regression.git>

8. Conclusion

Ce projet m'a permis de mettre en place une régression logistique complète en Python, sans utiliser de bibliothèque spécialisée en apprentissage automatique. Le modèle a été construit progressivement, en passant par toutes les étapes importantes : préparation des données, normalisation, fonction sigmoïde, calcul de la log-loss, descente de gradient, prédiction, évaluation et interprétation des résultats.

Les résultats obtenus montrent que le modèle fonctionne correctement sur le jeu de test. La baisse de la fonction de coût au fil de l'entraînement montre que l'algorithme apprend bien à partir des données. Les différentes métriques calculées à la main permettent aussi d'évaluer le modèle de façon plus complète.

La principale limite de ce travail vient du fait que les données utilisées sont artificielles. Pour aller plus loin, il serait intéressant d'appliquer la même implémentation sur un vrai jeu de données de classification binaire, par exemple dans le domaine médical ou universitaire.

Bibliographie

1. URENA, Raquel. Projet : Implémentation et Explication d'un Modèle de Régression Logistique en Python. Projet Mathématiques-Informatique
2. Wikipedia. Régression logistique. Disponible en ligne : https://fr.wikipedia.org/wiki/R%C3%A9gression_logistique
3. RASCHKA, Sebastian. Logistic Regression from Scratch. Disponible en ligne : https://sebastianraschka.com/Articles/2014_logistic_regression.html
4. CORMEN, Thomas H., LEISERSON, Charles E., RIVEST, Ronald L., STEIN, Clifford. Introduction à l'algorithmique. Pearson Education France, 2e édition, 2004.
5. THAIS, L., CASSAR, J.-P., FOULON, J. Rédaction d'un mémoire de stage ou de projet. École Polytechnique Universitaire de Lille.